Time-Dependent Decision Making in Heuristic Chess Search

Bachelor Thesis

submitted by

Marvin Becker (01/1365135)

at the

Algorithmics

Department of Computer and Information Science

Advisor: Prof. Dr. Sabine Storandt

Reviewer: Prof. Dr. Sabine Storandt
Prof. Dr. Michael Grossniklaus

Konstanz, 2026-09-01

Abstract

Chess engines rely heavily on heuristic search techniques to cope with the exponential growth of the game tree. Among these techniques, iterative deepening combined with alpha-beta pruning has become a standard approach due to its ability to produce strong moves under strict time constraints.

This thesis presents the design and implementation of a modular chess engine and investigates the relationship between search time and decision quality. The central question is where the sweet spot between invested time and move quality lies: how much additional search time actually translates into better decisions, and at which point further computation no longer changes the chosen move.

The engine is implemented from scratch in `C++` using a classical mailbox-based board representation. Its baseline integrates established techniques such as alpha-beta pruning, transposition tables, quiescence search, heuristic move ordering, and a two-stage time management system with soft and hard deadlines. On top of this baseline, the thesis develops two further search improvements: Principal Variation Search (PVS), which verifies all but the presumed best move with inexpensive null-window searches, and the reuse of search trees from previous searches, realized through a persistent transposition table that allows the engine to resume earlier computations.

The experimental evaluation studies time-dependent decision making along four axes: move stability, i.e., the depth at which the best move stabilizes and no longer changes with deeper search; time-to-quality curves, which capture how move quality evolves under increasing time limits; depth-versus-time behavior with and without Principal Variation Search; and a PVS parameter sweep that determines which combination of scouting depth and number of fully searched moves yields the best ratio of cost reduction to move quality.

The findings aim to characterize how gracefully decision quality degrades under tight time budgets and to identify configurations in which a deliberately simple engine architecture converts limited search time into stable, high-quality decisions.

Contents

1 Introduction

Chess has long been regarded as a canonical benchmark problem in artificial intelligence and algorithm engineering. Despite its simple and fully specified rules, the game exhibits an enormous search space caused by its high branching factor and strategic depth. As a consequence, exhaustive search of the complete game tree is computationally infeasible, even with modern hardware, and chess engines must rely on heuristic search techniques to make strong decisions under limited computational resources.

At the core of most classical chess engines lies a depth-limited game tree search based on Minimax or Negamax, enhanced by Alpha-Beta pruning. Over the years, a variety of improvements have been proposed to increase search efficiency, including iterative deepening, transposition tables, quiescence search, and heuristic move ordering. Among these techniques, iterative deepening has become a standard approach, particularly in time-controlled settings, as it allows the engine to gradually refine its decision while always maintaining a valid best move.

The primary motivation of this work is to understand how a chess engine should spend its limited time. More search time generally yields deeper searches and better moves, but the relationship is far from linear: at some point the chosen move stabilizes and additional computation no longer changes the decision. At the same time, search refinements such as Principal Variation Search and the reuse of previous search trees change how efficiently a given time budget is converted into depth and decision quality. This leads to the central research question of this thesis:

Where lies the sweet spot between invested search time and decision quality in heuristic chess search, and how do search refinements such as Principal Variation Search and search tree reuse shift this trade-off?

To make this question measurable, the thesis examines four concrete aspects: (i) *move stability*, i.e., the depth at which the best move stabilizes and deeper search no longer changes the decision, (ii) the *time-to-quality curve*, i.e., how move quality changes as the time limit increases, (iii) *depth-versus-time* behavior with and without Principal Variation Search, and (iv) a *PVS parameter sweep* that determines which combination of minimum scouting depth and number of fully searched moves yields the best ratio of cost reduction to move quality.

To address this question, a complete chess engine is implemented from scratch in `C++`. The engine employs a classical mailbox-based board representation rather than highly optimized bitboard techniques, which keeps the impact of search algorithms and time management observable in isolation from low-level optimizations; the underlying design rationale is discussed in **??**.

The implemented engine integrates a Negamax search with Alpha-Beta pruning, iterative deepening, quiescence search, transposition tables, and heuristic move ordering. A dedicated time management module supports multiple search modes, including fixed depth, fixed time, node-limited, and infinite search, following the UCI protocol. In particular, a two-stage time budget with soft and hard deadlines is used to control the interaction between iterative deepening and time constraints.

Beyond this baseline, this thesis develops two further search improvements: Principal Variation Search and the reuse of search trees from previous searches through a persistent transposition table. Both techniques are introduced formally in **??** and described at the implementation level in **??**.

The contribution of this work is threefold. First, it provides a clean and modular implementation of a classical chess engine that serves as a transparent experimental platform. Second, it extends the baseline search with Principal Variation Search and transposition-based search tree reuse and describes how these

techniques are realized in practice. Third, it presents an empirical evaluation of time-dependent decision making, analyzing move stability across depths, time-to-quality curves, depth-versus-time behavior with and without PVS, and the effect of the PVS tuning parameters. The results aim to locate the practical sweet spot between search time and decision quality in time-controlled chess engines.

The remainder of this thesis is structured as follows. **??** discusses relevant prior work in computer chess and game tree search. **??** introduces the necessary theoretical background. **??** describes the architecture of the engine and the implementation of the baseline search, the Principal Variation Search, the reuse of previous search trees, the time management, and the evaluation function. **??** presents the experimental evaluation, and **??** concludes with a discussion of the results and directions for future work.

2 Related Work

The problem of efficient search in chess engines has been studied extensively over several decades. Early approaches were based on the Minimax algorithm, which provides an optimal decision strategy under the assumption of perfect play [**?**]. However, due to the exponential growth of the game tree, Minimax quickly becomes infeasible even at moderate depths.

Alpha-Beta pruning was introduced as a major improvement over plain Minimax, allowing large parts of the search tree to be pruned without affecting the correctness of the result [**?**]. Subsequent work showed that the effectiveness of Alpha-Beta pruning depends crucially on the order in which moves are explored. With perfect move ordering, the effective branching factor can be reduced dramatically, enabling much deeper searches within the same computational budget [**?**].

Iterative deepening has emerged as a standard technique to improve move ordering and to support time-controlled search. By repeatedly searching the game tree with increasing depth limits, iterative deepening ensures that a valid best move is always available while simultaneously refining the move ordering for deeper iterations. Despite the apparent redundancy caused by re-searching parts of the tree, it has been shown that the overhead is often compensated by improved pruning efficiency [**?**].

A detailed analysis of Alpha-Beta pruning efficiency in chess engines is presented by Nair [**?**]. The paper investigates how different move ordering heuristics and search enhancements affect the pruning behavior of Alpha-Beta search. In particular, it highlights the strong interaction between iterative deepening, transposition tables, and heuristic move ordering. The results indicate that improved move ordering can significantly reduce the number of explored nodes, even when additional search iterations are introduced.

Building directly on the observation that Alpha-Beta search is most effective with strong move ordering, several refinements of the basic algorithm have been proposed. Pearl introduced the Scout algorithm, which verifies promising moves with minimal-window tests before evaluating them exactly [**?**]. Marsland and Campbell transferred this idea to the Alpha-Beta framework in the form of Principal Variation Search [**?**], and Reinefeld later refined it into NegaScout, analyzing the conditions under which the additional re-searches pay off [**?**]. PVS is particularly effective in combination with iterative deepening and transposition tables, since both increase the probability that the first move searched is indeed the best one [**?**].

Transposition tables have further improved search efficiency by allowing engines to reuse previously computed results for positions reached via different move orders [**?**]. By storing evaluations together with depth information and bound types, transposition tables enable both direct cutoffs and enhanced move ordering. Their effectiveness is especially pronounced in combination with iterative deepening, where information from earlier iterations can be reused almost immediately [**?**]. Beyond single search invocations, a persistent transposition table also allows information to be carried over from one move to the next, effectively resuming previously explored search trees instead of recomputing them from scratch.

Quiescence search addresses another well-known limitation of depth-limited search, commonly referred to as the horizon effect. By extending the search in tactically unstable positions, quiescence search produces more reliable evaluations and has become a standard component of classical chess engines [**?**].

While modern state-of-the-art engines incorporate additional techniques such as bitboards, evaluation learning, and neural networks [**?**], the classical search principles discussed above remain fundamental.

This work builds upon these established ideas and focuses specifically on the relationship between invested search time and decision quality.

3 Preliminaries

This section introduces the fundamental concepts and definitions required to understand the remainder of this thesis. It establishes the formal game-theoretic model of chess, derives the associated search problem, and motivates the use of heuristic evaluation and time-constrained search. All concepts presented in this section are standard in the field of computer chess and provide a common basis for the subsequent chapters.

3.1 Chess as a Game-Theoretic Model

Chess is modeled as a deterministic, two-player, zero-sum game with perfect information. At any point in time, the complete game state is fully observable to both players, and no randomness is involved in the game mechanics.

A game state consists of a board configuration together with additional state information such as the side to move, castling rights, and en-passant availability. Players alternate turns by selecting legal moves, and the game terminates in a win, loss, or draw according to the rules of chess.

Formal Representation From a theoretical perspective, the game can be represented as a directed graph. Each node corresponds to a legal position, and each directed edge corresponds to a legal move transforming one position into another. Terminal nodes represent positions in which no legal moves are available, such as checkmate or stalemate, as well as positions that are declared drawn by the rules of chess.

Game Tree Interpretation For decision making, the directed graph induces a game tree rooted at the current position. Each level of the tree corresponds to one ply. Due to the high branching factor of chess, which typically lies between 30 and 40 in middlegame positions, the size of the game tree grows exponentially with depth. As a result, even moderately deep exhaustive search is computationally infeasible.

3.2 Search Problem Formulation

Given a current position, the task of a chess engine is to select a legal move that maximizes the expected outcome under the assumption of optimal play by both sides. In the classical formulation, outcomes are mapped to numerical values, for example win, draw, and loss.

This decision problem is commonly formulated as a game tree search problem, in which the engine explores a subset of the game tree to estimate the value of available moves. The root of the tree corresponds to the current position, while internal nodes represent positions reachable through sequences of legal moves.

Resource Constraints Due to the exponential growth of the search tree, it is impossible to evaluate all reachable positions. Consequently, the search must be limited by depth, time, or other computational resources. This limitation introduces approximation: the engine can no longer guarantee optimal play,

but instead aims to select the best move according to the information explored within the available budget.

Role of Approximation The quality of a move selected by a chess engine therefore depends on two main factors: the effectiveness of the search algorithm in selecting which parts of the tree to explore, and the accuracy of the evaluation function used to estimate the value of positions at the search frontier.

3.3 Minimax, Negamax, Quiescence and Alpha-Beta Pruning

The classical algorithmic solution to game tree search in two-player zero-sum games with perfect information is the Minimax algorithm. Minimax assumes that one player aims to maximize the evaluation value of a position, while the opponent aims to minimize it. Terminal positions are assigned fixed values corresponding to win, loss, or draw, while non-terminal leaf positions are evaluated heuristically.

The algorithm explores the game tree recursively and propagates values from the leaves toward the root by alternating between maximization and minimization steps. A detailed introduction to Minimax can be found in standard AI textbooks, such as Russell and Norvig [**?**].

Negamax Formulation In zero-sum games, the Minimax algorithm can be simplified using the Negamax formulation. Instead of explicitly distinguishing between maximizing and minimizing players, Negamax expresses all evaluations from the perspective of the side to move. The value of a position is defined as the negation of the value obtained by the opponent after making a move.

This formulation exploits the inherent symmetry of zero-sum games and allows the search to be implemented using a single recursive procedure. Negamax is functionally equivalent to Minimax but leads to more compact and uniform code, which is why it is widely used in practical chess engines.

Quiescence Search Depth-limited game tree search combined with static evaluation functions suffers from a well-known problem referred to as the *horizon effect*. Tactical events such as captures, checks, or promotions may occur just beyond the search depth and are therefore not accounted for by the evaluation function. As a result, the engine may incorrectly assess positions as stable even though significant tactical changes are imminent.

Quiescence search is a standard technique to solve this problem. Instead of evaluating a position immediately when the nominal search depth is reached, the search is selectively extended in positions that are considered tactically unstable. Typically, only forcing moves such as captures or checking moves are explored, while quiet moves are excluded.

The goal of quiescence search is to reach a *quiescent* position, that is, a position in which no immediate tactical exchanges are available and the static evaluation is therefore more reliable. Once such a position is reached, the evaluation function is applied and the resulting score is propagated back up the search tree.

In practice, quiescence search is commonly implemented using Alpha-Beta pruning with a so-called *stand-pat* evaluation, which represents the static evaluation of the current position before any tactical extensions are explored. If the stand-pat score already exceeds the current bounds, the node can be pruned early.

Quiescence search significantly improves evaluation stability in tactical positions and has become a standard component of classical chess engines. A comprehensive discussion of the horizon effect and quiescence search can be found in survey articles on game tree pruning [**?**].

Alpha-Beta Pruning Alpha-Beta pruning is an optimization technique that reduces the number of nodes that need to be explored by Minimax or Negamax without affecting the correctness of the final result. The technique was introduced by Knuth and Moore [**?**] and is a cornerstone of modern game tree search.

The algorithm maintains two bounds during search:

- α, the best score that the maximizing player can guarantee so far,
- β, the best score that the minimizing player can guarantee so far.

At the root of the search tree, α is initialized to $-\infty$ and β to $+\infty$. As the search progresses, these bounds are updated based on the values of explored child nodes. Whenever a node is encountered for which it can be shown that its value cannot improve the current result, the corresponding subtree is pruned and not searched further.

More concretely, during a maximizing step, the value of α is updated whenever a better move is found. If α becomes greater than or equal to β, further exploration of sibling nodes is unnecessary, as the minimizing player would avoid reaching this position. An analogous argument applies during minimizing steps.

Effectiveness and Influencing Factors In the worst case, Alpha-Beta pruning explores the same number of nodes as plain Minimax. However, with optimal move ordering, the effective branching factor is significantly reduced, allowing the search to reach roughly twice the depth of unpruned Minimax for the same computational effort.

The effectiveness of Alpha-Beta pruning therefore depends strongly on the order in which moves are explored. Techniques such as heuristic move ordering, transposition tables, and iterative deepening are commonly employed to improve this order and thereby increase pruning efficiency. These aspects are discussed in more detail in later sections of this thesis.

Principal Variation Search Principal Variation Search (PVS) is a refinement of Alpha-Beta pruning that exploits good move ordering even further [**? ?**]. The technique is based on the assumption that the first move examined at a node is usually the best one. Only this first move is searched with the full window $[\alpha, \beta]$. Every remaining move is first examined with a so-called *null window* (or zero window) $[\alpha, \alpha + 1]$, which cannot return an exact score but can cheaply prove whether the move has the potential to improve upon α.

If the null-window search fails low, the move is confirmed to be inferior to the current best move and no further work is required. If it fails high, the assumption was wrong: the move may be better than the principal variation, and the node must be *re-searched* with the full window to obtain its exact score. PVS therefore trades occasional re-searches for substantially cheaper verification of the majority of moves. With strong move ordering, fail-highs are rare and the technique reduces the overall search effort considerably [**?**]. Conversely, with poor move ordering, frequent re-searches can negate the savings,

which is why PVS is typically combined with iterative deepening, transposition tables, and heuristic move ordering.

For a comprehensive discussion of Alpha-Beta pruning and its variants in the context of game-playing programs, see Marsland [**?**].

3.4 Heuristic Evaluation

Most positions encountered during search are non-terminal and therefore cannot be assigned an exact game-theoretic value. Instead, they must be evaluated heuristically. An evaluation function assigns a numerical score to a position, indicating its desirability from the perspective of the side to move.

Classical evaluation functions combine material considerations with positional features such as piece activity, king safety, and pawn structure. These features are typically handcrafted and encode domain knowledge about the game of chess. The design of the evaluation function has a significant impact on the playing strength of the engine, particularly when search depth is limited.

3.5 Time-Constrained Search

In practical settings, chess engines are required to operate under strict time constraints. Rather than searching to a fixed depth, the engine must dynamically adapt its search behavior to the available time budget.

Iterative Deepening

Iterative deepening is a widely used technique to address this requirement. The search is performed repeatedly with increasing depth limits, starting from a shallow search. This approach ensures that a valid move is always available, even if the search is interrupted prematurely.

Interaction with Pruning

In addition to providing robustness under time constraints, iterative deepening also improves the effectiveness of Alpha-Beta pruning. Information gathered in earlier iterations, such as good move orderings, can be reused in deeper searches, leading to a significant reduction in the number of explored nodes.

Search Tree Reuse Across Searches

The same principle extends beyond a single search invocation. After the engine has played a move and the opponent has responded, the new search root lies within the tree that was explored during the previous search. If the results of that search are retained, for example in a transposition table, the engine can *resume* the old search tree: previously computed scores yield immediate cutoffs, and previously identified best moves improve the move ordering from the very first iteration. Conceptually, the engine no longer starts each search from scratch but continues the computation it has already begun in earlier moves of the game.

The interaction between heuristic evaluation, Alpha-Beta pruning, and time-constrained iterative deepening forms the foundation of modern chess engines and is a central aspect of this work.

3.6 Universal Chess Interface (UCI)

In practice, chess engines are controlled through standardized communication protocols. The most widely used protocol in modern computer chess is the *Universal Chess Interface* (UCI).

UCI specifies a text-based command interface that allows external tools to set up positions, initiate searches, and receive search results. In particular, search constraints such as time limits, depth limits, node limits, and pondering are communicated via the `go` command.

The protocol itself is independent of the engine's internal implementation. As a result, UCI naturally separates external control from internal search logic. This separation is reflected in the design of the engine presented in this work, where UCI parameters are first mapped to an abstract constraint representation before being enforced during search.

All experiments reported in this work are conducted using UCI-compatible interfaces and tournament frameworks. UCI therefore provides the practical context in which time-constrained search and iterative deepening are evaluated.

4 Algorithm & Implementation

This section describes the algorithms used by the engine and how they were implemented. It first introduces the overall engine architecture and the baseline search, consisting of Negamax with Alpha-Beta pruning, iterative deepening, quiescence search, transposition tables, and heuristic move ordering. Building on this baseline, the two main algorithmic improvements of this thesis are presented: Principal Variation Search and the reuse of search trees from previous searches. The section concludes with the time management scheme and the static evaluation function.

4.1 Engine Architecture

This section describes the overall architecture of the implemented chess engine. The focus lies on the structural design decisions and the interaction between the core components; the rationale behind these decisions is summarized at the end of the section.

4.1.1 Overall Design

The engine follows a layered architecture that separates game representation, move generation, search, evaluation, and search control. Each component has a clearly defined responsibility and communicates with other modules through narrow and explicit interfaces.

?? provides a high-level overview of the engine structure and the interaction between the individual modules.

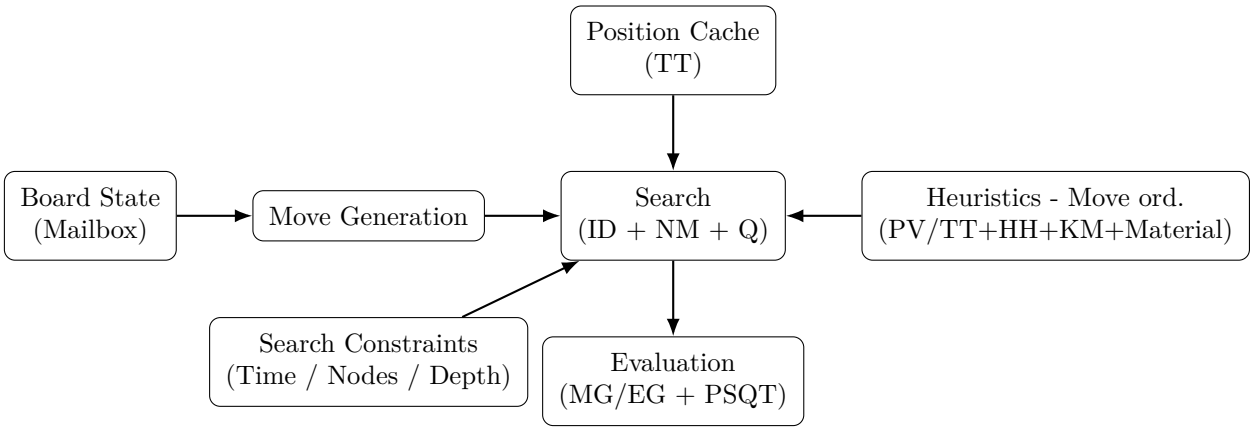

Figure 1: High-level architecture of the implemented chess engine.

At a conceptual level, the engine consists of the representation of the game state, the generation and application of moves, the search procedure, the static evaluation, and a unified constraint mechanism that controls search termination. This separation ensures that individual components can be modified or replaced without affecting the rest of the system, which is particularly important for experimental analysis.

4.1.2 Game State and Move Handling

Board Representation The board is represented using a classical mailbox model consisting of 64 squares. Each square explicitly stores the occupying piece or indicates that the square is empty. In addition to the piece placement, the board state contains information about the side to move, castling rights, king squares and en-passant availability.

Modern state-of-the-art engines typically rely on bitboard-based representations for efficient move generation and evaluation. However, mailbox representations remain a common choice in educational and experimental engines due to their simplicity and transparency [**?**]. Given the focus of this work on algorithmic behavior rather than raw performance, the mailbox model provides a clear and easily analyzable foundation.

Move Representation Moves are represented as explicit objects containing all information required to apply and undo them. This includes the source and destination squares, the moving piece, captured pieces, promotion information, and special move flags for castling, promotion and en-passant captures.

This design enables a fully reversible make–unmake mechanism, which is essential for recursive search algorithms. By storing all relevant information directly in the move object, the engine avoids recomputation when restoring previous board states, thereby simplifying the search implementation and reducing potential sources of errors.

Move Generation Move generation follows a direct and exhaustive approach. For each piece on the board, all potential target squares are examined according to the rules of chess. No precomputed attack tables or bitboard-based optimizations are used.

Illegal moves, such as those that leave the king in check, are filtered during move execution rather than during generation. This design keeps the move generator independent of check detection logic and makes its behavior easier to reason about. Although this approach is less optimized than state-of-the-art alternatives, it provides a robust and well-understood baseline for search experimentation.

4.1.3 Position Hashing and Transposition Tables

Zobrist Hashing To uniquely identify board positions, the engine employs Zobrist hashing. Each combination of square and piece type is associated with a random 64-bit value, and the hash of a position is computed by XOR-combining the values corresponding to the pieces currently on the board. Additional state information, such as the side to move, castling rights, and en-passant availability, is also incorporated into the hash.

Zobrist hashing is commonly used in game tree search because it allows board positions to be mapped efficiently to fixed-size keys. While the technique supports incremental updates during make–unmake operations, the implementation used in this engine recomputes the hash from the board state directly. The technique itself was originally introduced by Zobrist [**?**] and has since become a standard component of modern chess engines.

Transposition Table Usage The computed hash values are used as keys in a transposition table, which stores previously evaluated positions. Each entry contains the evaluation score, the search depth at which the position was evaluated, a bound type (exact, lower bound, or upper bound), and the best move found.

The transposition table serves two closely related purposes. First, it reduces search redundancy by reusing results for positions that can be reached via different move orders, which is common in chess. Without such a mechanism, identical positions would be evaluated repeatedly, leading to a significant increase in search effort. Second, transposition table entries improve move ordering by prioritizing moves that proved effective in previous searches. A simple aging mechanism ensures that outdated entries are gradually replaced, preventing the table from being dominated by stale information. Since the table persists across search invocations, it additionally serves as the mechanism by which the engine resumes previously explored search trees, as described in **??**. For practical implementations and further discussion, see for example Warnock [**?**].

4.1.4 Search Control and Integration

Unified Search Constraints All termination conditions for a search invocation are encapsulated in a single constraint structure. This includes depth limits, node limits, time budgets, and special modes such as infinite search or pondering.

Encapsulating all stopping criteria in a unified constraint object avoids scattered stop checks throughout the search code and ensures consistent behavior across different search modes. This design is particularly important for experimental analysis, as it guarantees that differences in search behavior are caused by the chosen constraints rather than by implementation artifacts.

Engine Integration The engine core is coordinated by a game management component that maintains the current position, applies and reverts moves, and invokes the search procedure. This component acts as a bridge between the external UCI interface and the internal search and evaluation modules, ensuring a clean separation between protocol handling and algorithmic logic.

4.1.5 Design Rationale

The architectural decisions made in this engine deliberately prioritize clarity, correctness, and experimental flexibility over maximum playing strength. Many state-of-the-art engines employ highly optimized data structures and aggressive micro-optimizations, which can obscure the underlying algorithmic behavior.

By choosing a modular and explicit design, this engine allows the effects of search algorithms, heuristics, and time management strategies to be studied in isolation. This aligns with the primary goal of this work, which is to analyze time-dependent decision making rather than to compete directly with top-performing chess engines.

4.2 Search Algorithms

This section describes the concrete implementation of the engine's search procedure. The underlying algorithms (Negamax, Alpha-Beta pruning, and quiescence search) were introduced in **??**; here, the focus is on how these ideas are realized in code and how the individual mechanisms interact in practice. Particular attention is paid to the two refinements of this thesis: the move loop is organized as a Principal Variation Search (**??**), and previous search trees are resumed through the persistent transposition table (**??**).

— 4.2.1 Search Entry Point and Modes

The main entry point is `ChessBot::think(Board board, SearchConstraints config)`. The `SearchConstraints` object is copied intentionally and stored inside the engine instance for the duration of the search. Before searching, the engine initializes a clean state via `reset_search_state()`, which resets node counters, selective depth tracking, stop state, and heuristic tables (killer and history), and also advances the transposition table generation via `tt.new_search()`.

Depending on `config.mode_`, the engine selects one of the following execution strategies:

- **FixedDepth:** the time budget is explicitly disabled (`budget_ = {}`) to prevent time-based termination inside `hard_stop()`. The engine performs a single root search with the requested depth and returns the result.
- **NodeLimit / Infinite:** time-based termination is disabled and the engine runs iterative deepening until an external stop condition (or the node limit) is hit.
- **Normal / FixedTime (time-managed):** the `TimeManager` initializes a soft and hard deadline from the constraints, and iterative deepening runs under these deadlines.

This structure keeps the actual search logic independent of UCI parsing and ensures that all termination behavior is expressed through the constraint object and the common stop checks.

— 4.2.2 Termination and Safety Checks

All immediate termination decisions are centralized in `ChessBot::hard_stop()`, which is called frequently inside the search (including quiescence) and combines an external stop flag, the hard time deadline, and an optional node budget. The exact semantics of these criteria, and the distinction between the hard deadline and the *soft* deadline that is only consulted between completed iterations of iterative deepening, are described in **??**.

As an additional robustness measure, the engine verifies the final move before returning it. If the selected move is not legal (checked via `is_legal_by_make_unmake()`), the engine falls back to a legal move obtained from `get_legal_fallback_move()`.

4.2.3 Iterative Deepening Loop

Time-managed searches run in `ChessBot::iterative_deepening(Board&)`. The engine starts with a legal fallback move and then performs repeated searches with increasing depth $d = 1, 2, 3, \ldots$.

For each iteration, a local copy of the current best move is passed into `root_search()`. If the search completes successfully, the resulting move becomes the new best move and is reported via the UCI `info` output (`print_info`). If an iteration is aborted (e.g., due to a hard stop), the loop terminates and the engine returns the best move from the last completed iteration.

Hard stops are enforced immediately inside the search, while soft stops only take effect at iteration boundaries (cf. **??**).

4.2.4 Root Search and Core Negamax

The helper `root_search(Board&, depth, Move&)` is a thin wrapper that invokes Negamax with the initial window $[-\infty, +\infty]$ and ply 0. The best move is returned through the referenced move parameter, which avoids additional return-value plumbing and keeps the recursive signature compact.

The core search routine `ChessBot::negamax(...)` implements a standard depth-limited Negamax with Alpha-Beta bounds:

- If `depth <= 0`, the search transitions into quiescence search.
- At every node, `hard_stop()` may terminate the search early.
- The routine maintains and updates α during move exploration; a cutoff occurs when $\alpha \geq \beta$.

To support correct transposition table storage, the implementation stores the original alpha value (`ORIGINAL_ALPHA`) and uses it later to classify the result as `EXACT`, `LOWER_BOUND`, or `UPPER_BOUND`.

Legal Move Handling Moves are generated as pseudo-legal moves and filtered by attempting to apply them with `board.make_move(move)`. If `make_move` fails (e.g., because the move leaves the king in check), the move is discarded. This design keeps move generation simpler and delegates legality to the make–unmake layer, which also guarantees that the search only recurses into legal positions.

Terminal Positions If no legal move exists, the implementation distinguishes:

- **checkmate:** if the side to move is in check, return a mate score of the form $-(MATE - ply)$,
- **stalemate:** otherwise return 0.

Using the current ply in the mate score ensures that faster mates are preferred and slower mates are avoided.

4.2.5 Principal Variation Search

The move loop inside `negamax` implements the Principal Variation Search scheme introduced in **??**. Instead of searching every move with the full window $[\alpha, \beta]$, the engine searches only the leading moves with the full window and verifies all remaining moves with a null-window search $[\alpha, \alpha + 1]$ (realized as $[-\alpha - 1, -\alpha]$ in the Negamax formulation). Only if the null-window search indicates that a move can beat α without already exceeding β is the move re-searched with the full window to obtain its exact score.

The behavior is controlled by two tunable parameters bundled in a `PvsConfig` structure:

- `min_depth` (default 2): below this remaining depth the scout search is skipped entirely and all moves are searched with the full window. At shallow nodes the subtrees are so small that the potential re-search overhead would outweigh the savings of the null-window search.

- `scout_after_move` (default 1): the first N legal moves of a node are always searched with the full window. Move ordering is least reliable for the very first moves, so the engine only switches to the cheap null-window scout once the presumed principal variation has been established.

Both parameters are exposed as UCI options (`PvsMinDepth` and `PvsScoutAfterMove`), which allows the scouting behavior to be tuned and evaluated experimentally without recompiling the engine. The overall structure of the move loop is summarized in **??**.

Algorithm 1: Principal Variation Search inside the Negamax move loop.

Input: Position P, depth d, bounds α, β, legal move counter n
Output: Score of the searched move

```
if n < scout_after_move or d < min_depth then
    score ← −NEGAMAX(P, d − 1, −β, −α)                    // full window
else
    score ← −NEGAMAX(P, d − 1, −α − 1, −α)                // null-window scout
    if α < score < β then
        score ← −NEGAMAX(P, d − 1, −β, −α)                // re-search
return score
```

A subtle but important implementation detail is the interaction with aborted searches: if a null-window scout is aborted by a hard stop, its score is not trusted and no re-search is triggered; instead, the abort is propagated upward so that the iterative deepening loop can discard the incomplete iteration.

4.2.6 Quiescence Search

Quiescence search is implemented in `ChessBot::quiescence(...)` and is entered whenever the regular search reaches depth 0. The function evaluates the current position via `eval::evaluate(board)` and then selectively extends the search by exploring only tactical moves (captures and en-passant). Non-capturing moves are skipped explicitly.

The implementation uses the standard *stand-pat* scheme:

- if the static score already exceeds β, the node fails high,

- otherwise α is updated using the static score and improved through capture exploration.

As with the main search, `hard_stop()` is checked inside quiescence to enforce hard termination constraints consistently.

4.2.7 Transposition Table Integration

The search consults the transposition table early in `negamax` via `tt.probe(key, depth, alpha, beta, ply, ...)`. A successful probe may:

- return an `EXACT` score,
- produce a safe cutoff using stored `LOWER_BOUND` or `UPPER_BOUND` information.

In addition, the table provides a stored best move (`tt_move`) which is used for move ordering. Since transposition table entries may be stale or collide, the implementation verifies the move's legality before using it; illegal TT moves are discarded.

After searching a node, the result is stored via `tt.store(...)` together with the depth, bound type, and the best move found at that node. Mate scores are encoded in a TT-safe form using the ply (via `to_tt_score` / `from_tt_score`) so that mate distances remain consistent across different search depths.

4.2.8 Search Tree Reuse Across Searches

The transposition table is not only used within a single search invocation but also serves as the engine's mechanism for resuming previous search trees. The table is a member of the engine instance and is deliberately *not* cleared between searches. When a new search starts, `reset_search_state()` only calls `tt.new_search()`, which advances an internal generation counter; all stored entries remain accessible. A full `tt.clear()` is performed only when the UCI command `ucinewgame` signals that an entirely new game begins.

This design directly implements the idea of continuing an earlier computation. After the engine has played a move and the opponent has answered, the new root position lies inside the tree that was explored during the previous search. Probing the persistent table therefore immediately recovers large parts of the earlier work:

- **Score reuse:** entries stored with sufficient depth yield exact scores or safe bound cutoffs without re-searching the corresponding subtrees.
- **Move ordering reuse:** stored best moves from the previous search guide the move ordering of the new search from the very first iteration, which is exactly the situation in which Principal Variation Search and Alpha-Beta pruning are most effective.

The generation counter implements a soft aging scheme rather than hard invalidation. The replacement policy prefers overwriting entries from older generations and entries with lower depth, so stale information gradually disappears as the game progresses while still being available as long as it is not displaced. In combination with iterative deepening, this means consecutive searches behave less like independent computations and more like one continuous search that is deepened and re-rooted as the game advances.

4.2.9 Move Ordering and Heuristics

Move ordering is implemented in `search::heuristics::order_moves`. The ordering score combines several standard heuristics:

- **TT/PV move first:** if a transposition table move exists, it receives a large bonus and is sorted to the front.
- **Captures before quiet moves:** captures are prioritized using a MVV-LVA-inspired score (most valuable victim / least valuable attacker), with a special case for en-passant.
- **Killer moves:** for each ply, the engine tracks up to two quiet moves that previously caused beta cutoffs and boosts them in ordering.
- **History heuristic:** a 3D table indexed by side, from-square, and to-square accumulates bonuses (scaled by depth squared) for quiet moves that caused cutoffs, and the accumulated score is added during ordering.

Killer and history updates are performed only on beta cutoffs and only for quiet moves. This avoids polluting these tables with captures and ensures that the heuristics reflect moves that are generally useful for pruning rather than tactical one-offs.

4.2.10 Debugging and Instrumentation

The search provides UCI-compliant reporting via `print_info`, including depth, selective depth, time, nodes, NPS, and either centipawn score or mate distance (converted from the internal mate representation). Additionally, a configurable debug facility can emit search health statistics, transposition table statistics, root move ordering, and principal variation output. This instrumentation supports the experimental goal of this thesis by making search behavior observable and reproducible.

4.3 Time Management and Constraints

In practical chess engines, search is not only limited by depth but often by a set of external constraints such as time, node budgets, or explicit stop signals. In this engine, all stopping conditions for a single search invocation are bundled in an immutable constraint object, which is initialized once before the search starts and treated as read-only during the search.

A key design goal is that the search code itself does not depend on UCI parsing details. Instead, the UCI layer produces a fully specified `SearchConstraints` instance, and the search only consults this object (and a stop flag) to decide when to terminate.

4.3.1 Unified Constraint Model

The engine uses a dedicated `SearchConstraints` structure to describe how a search is controlled and when it should terminate. This structure unifies all search modes under a single interface and avoids scattered termination logic across the search code.

A constraint instance contains:

- a **search mode** determining which criteria are active,
- an optional **fixed move time** (`movetime_ms_`),
- an optional **fixed depth** (`depth_`),
- an optional **node limit** (`nodes_`),
- and a **time budget** object (`budget_`) used for time-managed searches.

To simplify checks in the search, convenience predicates such as `has_fixed_depth()`, `has_node_limit()`, and `has_movetime()` are provided.

4.3.2 Search Modes

The active search behavior is selected via `SearchType`. The engine supports the following modes:

- **Normal:** Tournament-style time management with iterative deepening (default UCI behavior). Time allocation is derived from the clock settings and stored in `budget_`.
- **FixedTime:** Search for a fixed amount of time specified by `movetime_ms_` (UCI `go movetime`).
- **FixedDepth:** Search to a fixed depth only, determined by `depth_`, without relying on time control.
- **NodeLimit:** Search until a fixed number of nodes specified by `nodes_` has been explored.
- **Infinite:** Search indefinitely until an external stop command is received (UCI `go infinite`).
- **Ponder:** Search during the opponent's time. The engine continues searching until it is either stopped or the predicted opponent move is played, at which point the search can be continued or restarted.

This mode-based design enables a clean separation between the UCI interface (which interprets the `go` command) and the search itself (which only consults the active constraints).

4.3.3 Time Budgets: Structure and Arming

Time-based searches (Normal / FixedTime) rely on `TimeControl` stored in `SearchConstraints::budget_`. The budget distinguishes a *soft* and a *hard* limit:

- **Soft deadline:** preferred stopping point, checked only at safe boundaries (between completed iterations of iterative deepening).
- **Hard deadline:** strict upper bound that must not be exceeded, checked frequently throughout the search (including quiescence).

Unarmed Budgets for Non-time Modes A crucial implementation detail is that `budget_` is explicitly *disarmed* for non-time-based modes. Concretely, deadlines are set to zero, so `soft_time_up(...)` and `hard_time_up(...)` always return false. This ensures that depth-limited and node-limited search do not depend on any timing state.

Arming the Budget Before Search Immediately before a time-managed search begins, `TimeManager::init_search` is called. This function captures the current monotonic time (`now_ms()`) as the start timestamp and derives absolute deadlines:

$$\texttt{soft\_deadline\_ms\_} = \texttt{start\_ms\_} + \texttt{soft\_ms\_}, \qquad \texttt{hard\_deadline\_ms\_} = \texttt{start\_ms\_} + \texttt{hard\_ms\_}.$$

From this point onward, the budget object is treated as read-only.

4.3.4 Termination Conditions in the Search

The engine enforces termination using a combination of (i) an external stop flag, (ii) the hard deadline, and (iii) an optional node limit. These checks are centralized in `ChessBot::hard_stop()`, which is called frequently inside the main search and quiescence.

Hard Stop Checks The stop check follows the order shown in **??**: first an external flag, then the hard time deadline, then the node limit. If any condition is met, the search aborts immediately and returns a special "aborted" result to the caller.

Algorithm 2: Immediate termination check (`hard_stop()`).

Input: Current time *now*
Output: `true` iff the search must terminate immediately

```
if stop_requested is set then
    return true
if budget_.hard_time_up(now) then
    return true
if has_node_limit() and nodes >= nodes_ then
    return true
return false
```

Soft Stop in Iterative Deepening In contrast to the hard deadline, the *soft* deadline is checked only after a complete iteration has finished. This ensures that the engine does not stop mid-iteration, which would otherwise risk returning an unstable or partially explored root result. The structure is summarized in **??**.

4.3.5 Summary

The engine models time control and other stopping criteria using a single, immutable `SearchConstraints` object. By activating different termination rules depending on `SearchType` and by explicitly arming or

Algorithm 3: Soft stopping policy at iteration boundaries (iterative deepening).

Input: Board position P
Output: Best move found within the budget

```
bestMove ← legal fallback move
for d ← 1, 2, 3, ... do
    (score, aborted) ← depth-limited root search at depth d
    if aborted then
        break                                   // do not trust partial iteration
    bestMove ← move from completed iteration
    if budget_.soft_time_up(now) then
        break                                   // stop cleanly after a full iteration
return bestMove
```

disarming time budgets, the implementation remains robust and easy to analyze.

Most importantly, the implementation distinguishes between *hard* stopping conditions, which are enforced immediately inside the recursive search, and a *soft* stopping condition, which terminates the search cleanly between iterations of iterative deepening. This separation is essential for making time-constrained iterative deepening comparable to fixed-depth search in later experiments, since it ensures that the engine always returns the best move from the last fully completed depth.

4.4 Evaluation Function

This section describes the static evaluation function used by the engine. Since the search explores only a limited portion of the game tree, a heuristic evaluation is required to estimate the quality of non-terminal positions. The implemented evaluation follows a classical *tapered* approach with separate midgame (MG) and endgame (EG) scores, piece-square tables (PSQT), a phase-based interpolation scheme, and a tempo bonus.

4.4.1 Material Model

The evaluation assigns a material value to each piece type. Material is counted for both sides and is added to the positional contribution of the corresponding piece-square table. Conceptually, for a piece on square s, the score is:

$$\text{score}(p, s) = \text{material}(p) + \text{psqt}(p, s).$$

In the implementation, material values are used in two variants: a midgame value and an endgame value. This is consistent with the idea that the relative importance of pieces changes over the course of a game (e.g., kings become more active in endgames).

4.4.2 Piece-Square Tables (PSQT)

Positional preferences are encoded using piece-square tables. For each piece type, the engine stores a 64-entry table for midgame and endgame, assigning a positional bonus (or penalty) to each square. This

model captures common chess principles such as centralization, piece activity, and king safety without explicit feature engineering.

The implementation uses a single table orientation and mirrors squares for the white pieces by applying a rank flip. Concretely, for a board index i, white pieces use index $i \oplus 56$, while black pieces use i directly. This ensures that both colours share the same PSQT semantics relative to their respective perspective.

4.4.3 Midgame and Endgame Separation

Instead of a single score, the evaluation computes two scores:

- MG: midgame score (material + PSQT using MG tables),
- EG: endgame score (material + PSQT using EG tables).

The engine accumulates these values separately for both sides. The final score is then expressed from the perspective of the *side to move*. This is implemented by subtracting the opponent's accumulated values from the values of the player whose turn it is.

4.4.4 Game Phase Interpolation

To smoothly transition between midgame and endgame behavior, the engine uses a tapered evaluation. A game-phase value is computed as the sum of per-piece phase contributions. The resulting phase is clamped to a maximum of 24, yielding:

$$mgPhase = \min(24, \text{phase}(P)), \qquad egPhase = 24 - mgPhase.$$

The final evaluation is obtained by linear interpolation:

$$Eval(P) = \frac{MG(P) \cdot mgPhase + EG(P) \cdot egPhase}{24}.$$

This approach is commonly used in classical engines, as it avoids abrupt evaluation changes and provides a simple, interpretable blending scheme.

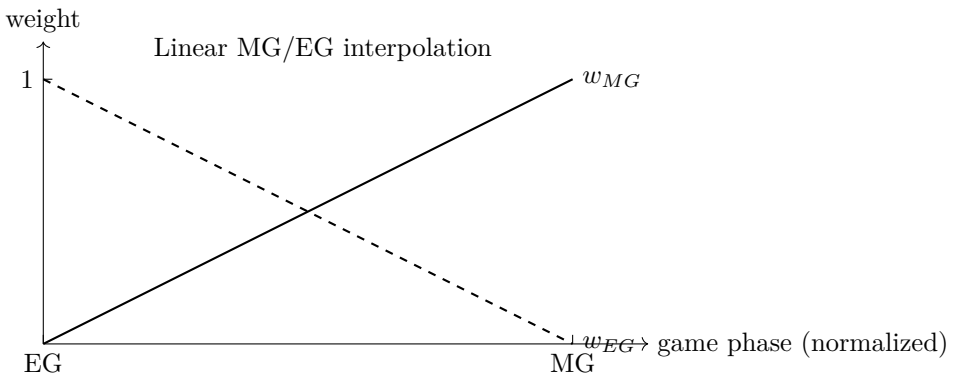

Figure 2: Tapered evaluation: midgame and endgame weights vary linearly with the game phase.

4.4.5 Tempo Bonus

A constant tempo bonus is added to the final score to reflect the advantage of having the next move. In the implementation, the returned evaluation is:

$$Eval_{\text{final}}(P) = Eval(P) + 20.$$

Since the engine evaluates positions from the perspective of the side to move, this constant shifts the score slightly in favor of the player whose turn it is, encouraging initiative and reducing symmetry artifacts.

4.4.6 Algorithmic Summary

Algorithm 4: Static evaluation with MG/EG separation, tapered interpolation, and tempo bonus.

Input: Board position P
Output: Static evaluation score from the perspective of the side to move

Initialize $mg[2] \leftarrow 0$, $eg[2] \leftarrow 0$, $phase \leftarrow 0$
for $i \leftarrow 0$ **to** 63 **do**
  $piece \leftarrow P[i]$
  $phase \leftarrow phase + phaseValue(piece)$
  if *piece is white* **then**
    $idx \leftarrow i \oplus 56$ `// mirror for white`
    $mg[W] \leftarrow mg[W] + MG\_PSQT(piece, idx) + MG\_MAT(piece)$
    $eg[W] \leftarrow eg[W] + EG\_PSQT(piece, idx) + EG\_MAT(piece)$
  else if *piece is black* **then**
    $idx \leftarrow i$
    $mg[B] \leftarrow mg[B] + MG\_PSQT(piece, idx) + MG\_MAT(piece)$
    $eg[B] \leftarrow eg[B] + EG\_PSQT(piece, idx) + EG\_MAT(piece)$
$mgPhase \leftarrow \min(24, phase)$
$egPhase \leftarrow 24 - mgPhase$
Compute $MG(P)$ and $EG(P)$ as difference between side-to-move and opponent
$Eval(P) \leftarrow \big(MG(P) \cdot mgPhase + EG(P) \cdot egPhase\big)/24$
return $Eval(P) + 20$ `// tempo`

4.4.7 Limitations of the Evaluation

The evaluation is intentionally simple and focuses on material and PSQT-based positional features. This makes it fast and easy to analyze, but it also implies several limitations:

- **Limited feature set:** Pawn structure (isolated/doubled/passed pawns), mobility, rook activity on open files, bishop pair, and advanced king safety concepts are not modeled explicitly.
- **No explicit drawishness detection:** The evaluation does not implement specialized scaling for fortress-like positions, opposite-coloured bishop endgames, or other draw-prone material configurations.

- **Constant tempo term:** The tempo bonus is a fixed constant and does not depend on phase or position characteristics.

- **No incremental evaluation:** The score is computed by scanning all 64 squares, rather than updating evaluation terms incrementally during make/unmake.

Despite these limitations, the evaluation is sufficient for the primary focus of this thesis, namely the investigation of search behavior and iterative deepening under time constraints. A stronger evaluation can be integrated later without changing the surrounding search architecture.

5 Experimental Evaluation

This section presents the experimental evaluation of the implemented chess engine. The evaluation pursues the central question of this thesis: where lies the sweet spot between invested search time and decision quality? To this end, it first establishes a reliable baseline for the playing strength of the engine and quantifies the effect of the search optimizations, and then studies time-dependent decision making directly through move stability, time-to-quality, and depth-versus-time measurements, including the influence of Principal Variation Search and its tuning parameters.

5.1 Research Objectives

The experimental evaluation addresses the following research questions:

- How strong is the optimized engine under realistic tournament time controls when compared to a fixed reference opponent?
- How large is the performance difference between optimized search and plain Negamax with Alpha-Beta pruning when evaluated on identical positions?
- **Move stability:** at which search depth does the best move stabilize, so that deeper search no longer changes the decision?
- **Time-to-quality:** how does move quality change as the time limit is increased, and where does additional time stop paying off?
- **Depth versus time:** how does the reachable depth per time budget differ with and without Principal Variation Search?
- **PVS parameter sweep:** which combination of minimum scouting depth (`PvsMinDepth`) and number of fully searched moves (`PvsScoutAfterMove`) yields the best ratio of cost reduction to move quality?

The first two questions establish the baseline strength and the effect of the search optimizations; the remaining four questions form the core of this thesis and characterize time-dependent decision making directly.

5.2 Engine-versus-Engine Evaluation

5.2.1 Experimental Setup

The engine under evaluation (*Helix*) was tested in head-to-head matches against a fixed reference opponent under controlled and reproducible conditions.

Engines. Helix was evaluated against a development version of Stockfish (`Stockfish dev-20251117-035cb146`). To obtain a meaningful comparison in the target strength range, Stockfish was configured using its built-in Elo limitation mechanism with `UCI_LimitStrength` enabled and a target strength of 2000 Elo. In addition, Stockfish was restricted to a single search thread and a hash size of 32 MB. Pondering was disabled for both engines.

Time Control. All games were played using a classical tournament time control of 5 minutes per side with an increment of 0.5 seconds per move. This setting ensures that time management and iterative deepening behavior have a significant influence on search quality while still allowing sufficiently deep searches.

Match Configuration. A total of 400 games were played using the CuteChess testing framework. No opening book was used; all games started from the standard initial position. Colors were alternated, and each engine played an equal number of games as White and Black to ensure fairness.

Hardware and Software Environment. All experiments were conducted on a MacBook Pro equipped with an Apple M3 Pro processor and 18 GB of RAM. Both engines were compiled in release mode and run as single-threaded processes. Apart from the explicitly stated options, all engine parameters were kept constant throughout the experiments.

Evaluation Methodology. Playing strength was measured using Elo difference estimates derived from match results. In addition to the Elo estimate, the likelihood of superiority (LOS) and the draw ratio were recorded to assess statistical confidence and game decisiveness.

5.2.2 Results

Table **??** summarizes the results of the match series between Helix and the reference Stockfish configuration.

Metric	Value
Games played	400
Elo difference	$+161.9 \pm 36.2$
Likelihood of Superiority (LOS)	100.0 %
Draw ratio	7.5 %

Table 1: Match results of Helix against Stockfish limited to 2000 Elo.

Out of the 400 games played, Helix achieved a clear majority of wins. The low draw ratio indicates that most games were decided decisively rather than through repetition or stalemate.

A breakdown of decisive outcomes shows that Helix won 279 games by checkmate (130 as Black and 149 as White), while Stockfish won 91 games by checkmate (38 as Black and 53 as White). The remaining 30 games ended in draws due to threefold repetition.

The estimated Elo difference of approximately +162 points in favor of Helix is statistically significant, as indicated by the 100 % likelihood of superiority. It should be emphasized that the reference engine was deliberately limited to 2000 Elo; the reported difference therefore reflects relative performance under this configuration rather than an absolute strength comparison.

5.3 Search Optimization Ablation Study

To isolate the effect of the implemented search optimizations, an additional controlled ablation experiment was conducted. In contrast to the engine-versus-engine evaluation, this experiment focuses exclusively on search efficiency under identical time constraints.

5.3.1 Experimental Design

The engine was evaluated in two configurations:

- **Optimized search:** Full implementation including transposition tables and heuristic move ordering (TT move, captures, killer moves, history heuristic).
- **Unoptimized search:** Plain Negamax with Alpha-Beta pruning, without transposition tables and without heuristic move ordering. Moves are explored in their generated order.

Both configurations use the same board representation, move generator, and evaluation function. Any observed differences can therefore be attributed directly to the presence or absence of search optimizations.

5.3.2 Positions and Time Control

The experiment was performed on a fixed set of 128 predefined chess positions. For each position, both engine configurations were given a fixed search time limit of 2 seconds. No depth or node limits were imposed.

This setup ensures that both configurations operate under identical external constraints and that observed differences reflect how efficiently the available time is converted into search depth and explored nodes.

5.3.3 Evaluation Metrics

For each position and configuration, the following metrics were recorded:

- total number of explored nodes,
- maximum search depth reached within the time limit,
- average search depth over all positions.

All results are aggregated across the full set of 128 positions.

5.3.4 Results

Table **??** compares the average number of explored nodes per search depth for the optimized and unoptimized configurations under the fixed 2-second time limit.

Depth	Optimized Nodes (avg.)	Unoptimized Nodes (avg.)
1	≈ 36	≈ 1651
2	≈ 114	≈ 278
3	≈ 625	≈ 2596
4	≈ 1689	≈ 19766
5	≈ 9410	≈ 185529

Table 2: Average number of explored nodes per depth under a fixed 2-second time limit, aggregated over 128 positions.

The optimized configuration consistently reaches greater search depths and explores significantly fewer nodes per depth level. This indicates a substantial reduction in the effective branching factor compared to the unoptimized search.

In a subset of the tested positions, both configurations selected identical best moves and principal variations despite differences in achieved search depth. A detailed discussion of this observation is provided in Section **??**.

6 Conclusions and Future Work

This section discusses the experimental results presented in Section **??**, relates them to the central research question of this thesis, and draws overall conclusions. The focus lies on interpreting the observed effects of search optimizations under time constraints and on understanding their implications for iterative deepening search in practical chess engines.

6.1 Search Efficiency and Effective Branching Factor

The ablation study clearly demonstrates that the implemented search optimizations lead to a substantial reduction in search effort under identical external constraints. When operating under a fixed time limit, the optimized search explores significantly fewer nodes per depth level than the unoptimized plain Negamax with Alpha-Beta pruning.

This behavior can be interpreted as a reduction of the effective branching factor. While Alpha-Beta pruning already reduces the theoretical branching factor compared to naive Minimax search, its practical effectiveness depends strongly on the order in which moves are explored. Without informed move ordering, large parts of the game tree must still be searched before cutoffs occur.

The optimized configuration benefits from a combination of transposition tables and heuristic move ordering, which prioritizes tactically and strategically relevant moves early in the search. As a result, Alpha-Beta cutoffs occur earlier and more frequently, allowing the engine to reach greater effective depth within the same time budget.

6.2 Depth versus Decision Quality

An important insight from the experiments is that search depth alone is an insufficient measure of decision quality. In a notable fraction of the tested positions, both the optimized and unoptimized configurations selected the same best move and, in some cases, identical principal variations, despite differences in achieved search depth.

This observation suggests that the optimized search often converges to decision-relevant lines earlier than the unoptimized search. In such situations, additional depth gained by an uninformed search does not necessarily translate into improved decision quality, but instead reflects redundant exploration of less relevant branches.

From an algorithmic perspective, this highlights the importance of directing search effort toward critical variations rather than maximizing depth in an uninformed manner. A shallower but better-focused search can therefore yield decisions of equal or higher quality than a deeper search with poor move ordering.

6.3 Implications for Iterative Deepening under Time Constraints

The results have direct implications for time-constrained iterative deepening search. Iterative deepening inherently trades additional overhead from repeated searches for improved move ordering and search

stability. The experimental findings indicate that this trade-off is beneficial when combined with effective search optimizations.

Under strict time limits, the optimized engine is able to preserve move quality even when the maximum achievable depth is reduced. This supports the central hypothesis of this work: degradation in playing strength under time constraints is not determined solely by reduced depth, but by how effectively the available time is converted into meaningful search.

Furthermore, the observed stability of best moves across different depths suggests that iterative deepening provides a robust mechanism for time-managed search. Even when interrupted before reaching maximal depth, the engine often returns a move that is consistent with deeper searches.

6.4 Relation to Practical Playing Strength

The engine-versus-engine evaluation complements the ablation study by confirming that improved search efficiency translates into higher practical playing strength. The optimized engine significantly outperforms the reference opponent under classical tournament time controls, despite operating under identical hardware and threading constraints.

While this result depends on the chosen reference configuration, it provides empirical evidence that the observed reductions in search effort and improvements in move quality have tangible effects on game outcomes. The combination of efficient pruning, informed move ordering, and stable time management enables the engine to allocate its computational resources more effectively throughout the game.

6.5 Limitations

Several limitations of the presented evaluation should be noted. First, move quality is assessed indirectly through agreement of best moves and principal variations rather than through comparison with ground-truth solutions or exhaustive searches. While this provides valuable insight into search behavior, it does not guarantee objective optimality.

Second, the ablation study is conducted under a fixed time limit and on a finite set of predefined positions. Although this setup allows controlled comparison, results may vary for different position distributions or time budgets.

Finally, the engine deliberately uses a transparent mailbox board representation and a handcrafted evaluation function. While these choices support clarity and experimental control, they may limit absolute playing strength and affect how well the conclusions generalize to state-of-the-art engines.

6.6 Summary

In summary, the discussion highlights that effective search optimizations improve not only raw search efficiency but also the reliability of decision-making under time constraints. The results support the view that in practical chess engines, search quality depends more on informed pruning and move ordering than on raw search depth alone. These findings motivate further investigation into time-managed search strategies and provide a solid foundation for future extensions of this work.